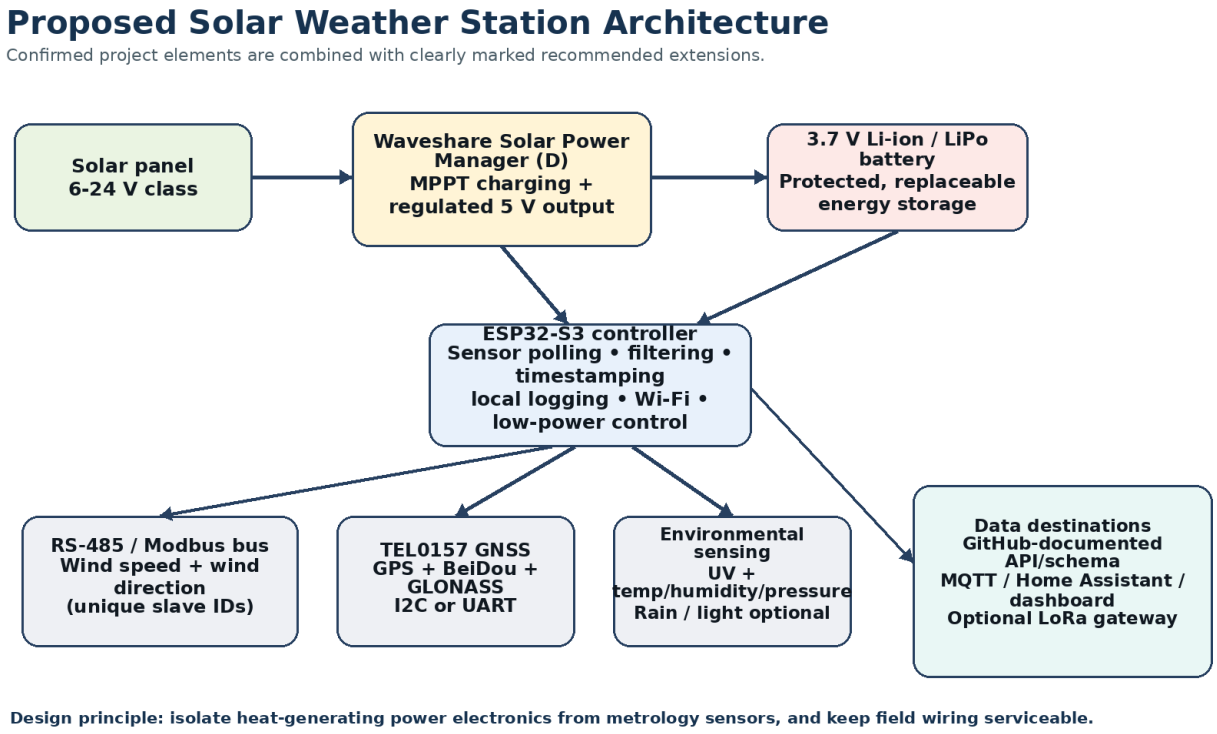
SOLAR WEATHER STATION

Open-Source Project Foundation & GitHub Repository Blueprint

Green Energy Projects

Version 0.1 | 14 August 2026



Purpose: a reproducible engineering baseline for building, documenting, testing, and publishing the project as an open-source reference implementation.

1. Executive Summary

This document consolidates the Solar Weather Station work developed inside the Green Energy Projects workspace into a single engineering baseline. It is intended to become the foundation for a public GitHub repository that another builder can understand, reproduce, modify, test, and improve.

The design is inspired by Open Green Energy's Solar Powered WiFi Weather Station V4.0, but the current project should be treated as an independent adaptation rather than a copy. The reference platform uses an ESP32, a custom solar-charging PCB, LoRa, BME280/DS18B20, SparkFun wind and rain instruments, SI1145 UV sensing, BH1750 light sensing, PMS5003/7003 particulate sensing, and SHT10 soil sensing. The present project has already moved toward an ESP32-S3 controller, RS-485/Modbus wind instruments, a DFRobot TEL0157 multi-constellation GNSS module, a Waveshare Solar Power Manager (D), and a custom 3D-printed enclosure/power compartment. [1][2] [6][7]

What this foundation establishes

- A clear problem statement and project purpose, including why a solar-powered local weather station is useful for microclimate monitoring and remote deployment.
- A traceable distinction between confirmed project choices, components inherited only from reference designs, and recommendations that are still optional.
- A modular electrical architecture around ESP32-S3, solar energy storage, RS-485/Modbus sensing, GNSS, environmental sensing, and network data publication.
- A practical budgeting model with prototype, outdoor, and ruggedized cost bands rather than a single misleading fixed price.
- A staged validation plan covering communications, sensor correctness, current consumption, solar autonomy, enclosure durability, and field data quality.
- A GitHub repository structure, release checklist, documentation strategy, and licensing notes suitable for an open-source hardware/software project.

Current project maturity

Subsystem	Current state	Foundation recommendation
Controller	ESP32-S3 selected / used in planning	Keep ESP32-S3 as the main controller; document the exact dev-board SKU before release.
Wind sensing	RS-485 wind speed and wind direction work has been developed; direction decoding is working	Place compatible Modbus devices on one shared differential bus with unique slave IDs.
RS-485 interface	MAX485 module used during diagnostics	For the final board, prefer a native 3.3 V or isolated RS-485 transceiver; MAX485 is a 5 V part. [12]
GNSS	DFRobot TEL0157 communicates; early diagnostics showed valid module communication while waiting for satellite fix	Mount antenna with sky view; power GNSS only when needed if energy budget requires it. [6]
UV	SI1145 and GUVA-S12SD considered	Do not make SI1145 the default new-

		build choice; it is no longer stocked by Adafruit and estimates UV from visible/IR. GUVA-S12SD is an analog true UV photodiode option; a modern digital true-UV part can also be offered. [8][9]
Power	Waveshare Solar Power Manager (D) considered/used in the power architecture	Use solar input + protected 3.7 V battery + regulated 5 V output to the ESP32-S3; document Type-C behavior and dedicated output wiring. [7]
Mechanical	Custom enclosure, louvers, battery/power container, cable-management and removable joining concepts developed	Separate metrology airflow from battery/power heat; use PETG/ASA, strain relief, captive fastening, and rain-baffled ventilation.

2. Document Map

3. Problem Identification and Project Purpose

3.1 Problem identification

Public weather services are designed to describe regional conditions. They are not always representative of the exact microclimate at a rooftop, farm plot, construction site, solar array, campus, greenhouse, or remote test location. A local weather station can measure the conditions where an engineering or operational decision is actually being made. The Open Green Energy V4.0 project frames the same need around smart agriculture, smart cities, solar plants, construction sites, and remote deployments. [1]

3.2 Engineering problem

The practical challenge is not simply to read sensors. The system must collect multiple weather variables outdoors while operating from intermittent solar energy, survive heat/rain/UV exposure, communicate over mixed interfaces, preserve data quality, and remain serviceable by people other than the original builder. In a future open-source release, reproducibility and maintainability are part of the engineering problem, not documentation afterthoughts.

3.3 Project purpose

The purpose of this project is to build a modular, solar-powered, internet-capable weather station using an ESP32-S3 as the main controller. The station should support reliable local weather measurements, self-powered remote operation, structured data logging, and an open hardware/software documentation package that can be reproduced by students, makers, researchers, and engineers.

3.4 Intended use cases

- Microclimate monitoring around renewable-energy installations.
- Smart agriculture and greenhouse support.
- Construction-site environmental monitoring.
- Educational IoT, embedded systems, renewable-energy, and 3D-design demonstrations.
- Field testing of low-power networking and sensor interfaces.
- A public GitHub case study showing the full engineering lifecycle from problem identification to validated prototype.

4. Objectives, Scope, and Success Criteria

4.1 Primary objectives

- Collect useful meteorological parameters with a modular sensor architecture.
- Operate from solar energy and rechargeable battery storage without continuous external power.
- Use robust wired interfaces for exposed field sensors, particularly RS-485/Modbus for wind instruments.
- Provide time/location context using GNSS when required.

- Publish or store measurements in a structured format that can feed dashboards, MQTT, Home Assistant, or a future cloud backend.
- Use a mechanically serviceable enclosure that protects electronics without thermally biasing weather sensors.
- Publish reproducible firmware, wiring, CAD exports, BOM, assembly instructions, test procedures, and known limitations.

4.2 Out-of-scope for the first public release

- Certified meteorological-grade accuracy.
- Formal IP-rating certification.
- Commercial product compliance/certification.
- Long-term unattended operation claims before seasonal field testing.
- A mandatory LoRa receiver node; LoRa should remain an optional transport until the core local station is stable.

4.3 Proposed measurable success criteria

Area	Release target
Sensor integration	All selected sensors produce stable readings for a continuous 24 h bench test without bus lockup.
RS-485	Wind devices respond reliably on one bus using documented baud/parity/register map and unique slave IDs.
GNSS	Valid date/time and position fix obtained outdoors; acquisition behavior documented.
Power	System runs from the solar-manager/battery path and automatically recovers after battery/solar interruptions.
Low power	Sleep/current behavior measured rather than assumed; current profile attached to the repository.
Outdoor test	At least 72 h outdoor run with no enclosure leak, loose wiring, reset loop, or corrupted data.
Reproducibility	A second build can be assembled using only repository files and the BOM.
Documentation	README, wiring, BOM, CAD, firmware build steps, sensor calibration notes, test log, and license all present.

5. Reference Project Lineage

5.1 Primary reference: Open Green Energy Weather Station V4.0

The earliest design reference in this project was the Elecrow page for “DIY Solar Powered WiFi Weather Station V4.0,” which points to the broader Open Green Energy V4.0 design lineage. The

Hackaday project describes a two-node architecture: a solar-powered sender in the field and a receiver located where internet access is available. Data is sent over LoRa and can be displayed locally or forwarded to services such as Home Assistant, ThingSpeak, or Blynk. [1]

The V4.0 reference design measures BME280 temperature/humidity/pressure, DS18B20 external temperature, SparkFun wind speed/direction/rain, SI1145 UV index, BH1750 light, PMS5003/7003 particulate matter, and SHT10 soil parameters. Its hardware also includes a solar/battery subsystem and low-power techniques such as ESP32 deep sleep and power switching for LoRa and sensors. [1] [2]

5.2 Related GitHub firmware

The Hackaday log links to James Hughes' stable transmitter and receiver firmware. The transmitter repository `jhughes1010/weather_v4_lora` is GPL-3.0 licensed; the receiver repository `jhughes1010/weather_v4_lora_receiver` is also GPL-3.0 licensed. [3][4]

A later community adaptation, `teamsuperpanda/weather-station`, converts the idea to ESPHome/Home Assistant and explicitly states that it is based on Open Green Energy V4.0. That repository was archived on 7 June 2026 and uses PolyForm Noncommercial 1.0.0, so it should not be used as the licensing model for a project that wants conventional open-source status. [5]

5.3 How the present project differs

Design topic	Open Green Energy V4.0	Current Green Energy Projects direction
MCU	ESP32-WROOM32-U	ESP32-S3 development board / module
Wind instruments	SparkFun weather meter analog/pulse interfaces	RS-485 / Modbus wind speed and wind direction devices
Wireless transport	LoRa sender → receiver; internet from receiver	Start with direct ESP32-S3 connectivity; keep LoRa as optional architecture
GNSS	Not a central V4.0 element	DFRobot TEL0157 GPS/BeiDou/GLONASS module
Power hardware	Custom V4.0 charging/power PCB	Waveshare Solar Power Manager (D) plus custom integration
Mechanical design	Stacked Stevenson-screen style enclosure	Custom triangular/louvered enclosure, separate battery/power container, removable upper assembly
Repository goal	Project instructions and code	Reproducible engineering case study with tests, CAD, BOM, decisions, risks, and roadmap

6. Project Design History and Decisions

The following timeline summarizes decisions and lessons from the available Solar Weather Station conversations in the Green Energy Projects workspace. It is deliberately written as an engineering traceability log so future repository readers can understand why the design evolved.

Date	Design activity / evidence	Status
26 May 2026	Reference project identified from Elecrow/Open Green Energy V4.0. SI1145 versus GUVA-S12SD UV sensing discussed. ESP32-S3 and 8×12 cm / 10×15 cm perfboard layouts considered. Mounting of the power board on perfboard and overall power-section integration discussed.	Baseline concept
26 May 2026	RS-485/MAX485 diagnostics performed for wind sensing. A raw UART “loopback” test with A and B tied together produced no received bytes.	Important diagnostic lesson
15 Jun 2026	TEL0157 GNSS communications succeeded (`gnss.begin() OK`), but the module initially had no satellite fix and reported zeroed position/time values.	Interface works; outdoor acquisition required
17 Jun 2026	Wind-direction register interpretation was explored; wind direction was later reported as working correctly.	Confirmed functional progress
17 Jun 2026	Need identified to change Modbus slave ID, enabling multiple RS-485 instruments to share a bus.	Architecture-enabling step
2 Jul 2026	Custom enclosure developed with three louvers on long triangular edges; hexagonal/vent geometry and printability discussed. Cable management, battery/power lower container, removable upper enclosure, and click/latch joining concepts explored.	Mechanical concept progressing
14 Aug 2026	Project consolidation requested as the basis for a public GitHub repository and future open-source demonstration.	Documentation milestone

7. Proposed System Architecture

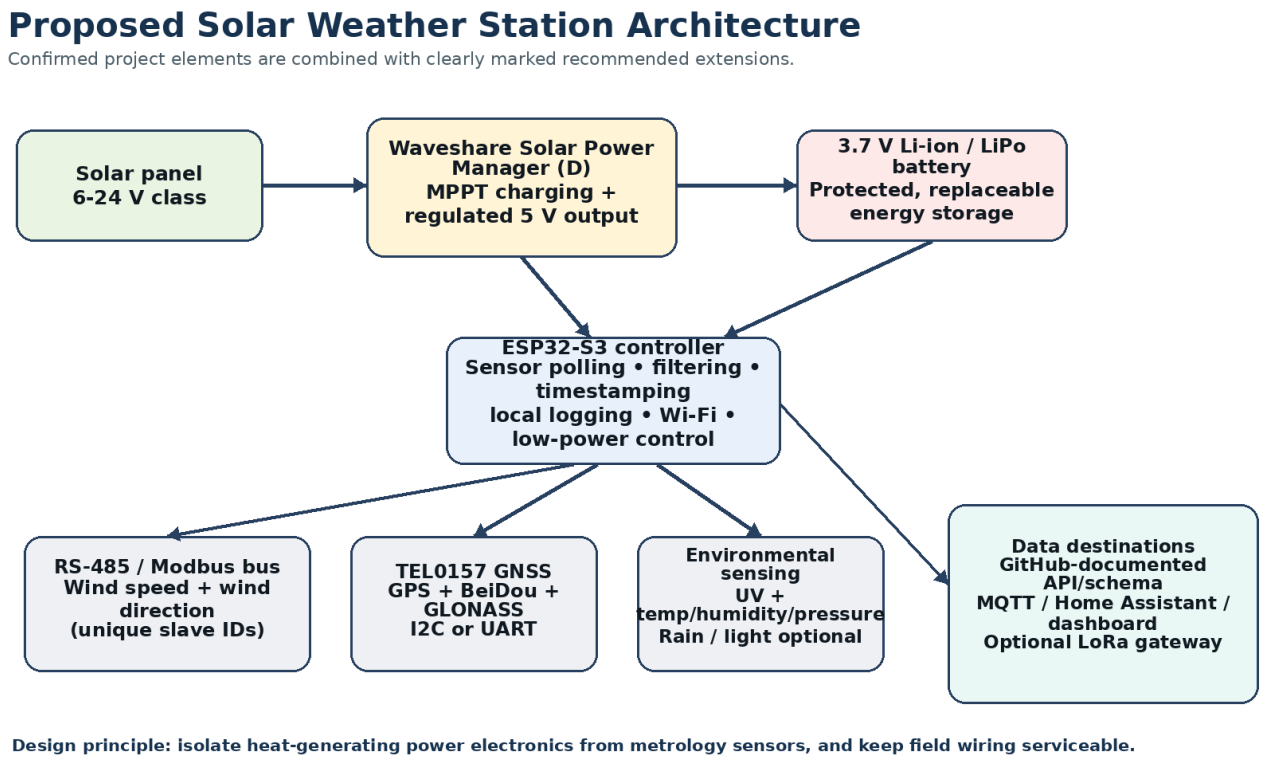


Figure 1. Proposed modular system architecture.

7.1 Architectural principle

Build the station as replaceable subsystems rather than one monolithic wiring harness. The MCU should not need to know the physical enclosure details, and the mechanical enclosure should not force a particular cloud platform. Each sensor driver should expose a common measurement structure with timestamp, value, units, validity, and diagnostic status.

7.2 Recommended logical flow

1. Wake or begin sampling cycle.
2. Enable sensor power rails if switched.
3. Poll RS-485 devices using known slave IDs and validated register maps.
4. Read environmental sensors and UV.
5. Acquire or reuse GNSS time/position according to the configured GNSS policy.
6. Validate ranges; attach sensor-health flags.
7. Store locally before transmission so network failure does not automatically mean data loss.
8. Publish over Wi-Fi/MQTT/HTTP; optional LoRa transport can be added as an alternative output adapter.
9. Enter low-power state when the sampling strategy allows it.

8. Hardware Subsystems

8.1 Main controller: ESP32-S3

CONFIRMED ESP32-S3 is the preferred main MCU for the current project direction.

ESP32-S3 provides 2.4 GHz Wi-Fi, Bluetooth LE, a dual-core processor, and a large set of programmable GPIO/peripheral interfaces. Espressif documents both light-sleep and deep-sleep modes; in deep sleep the main CPUs and most digital peripherals are powered off, which is important for solar operation. [10][11]

- Document the exact board/module and pinout in `hardware/controller/`.
- Do not reserve GPIOs informally; maintain a version-controlled pin map.
- Keep UART, I2C, ADC, interrupt-capable GPIO, and power-switch control requirements separate when assigning pins.
- Measure the development board's actual sleep current because USB bridges, regulators, and LEDs can dominate the SoC deep-sleep current.

8.2 Wind sensing: RS-485 / Modbus

CONFIRMED Wind direction has been successfully decoded in the project; wind sensing is being implemented with RS-485 instruments.

RS-485 is appropriate for exposed weather-station instruments because it is a balanced differential physical layer widely used with Modbus over long cable runs and electrically noisy environments. The project should use unique Modbus IDs for each sensor and one shared bus where the devices support the same serial settings. [12]

- Record manufacturer, model, supply voltage, Modbus mode, baud rate, parity, stop bits, slave ID, function code, register address, scale factor, units, and valid range for every device.
- Use twisted pair for A/B; provide a reference conductor/ground strategy appropriate to the sensor datasheet.
- Use termination at bus ends only when cable length/data rate justify it; avoid random termination resistors at every node.
- For outdoor/long-cable versions, add TVS/surge protection and consider galvanic isolation.
- Use a single address-change utility and keep the original/default and assigned IDs in the repository.

8.3 RS-485 transceiver choice

CONFIRMED MAX485 has been used in diagnostics.

The MAX485 is a single-5 V RS-485 transceiver. For a final ESP32-S3 design, a native 3.3 V transceiver is cleaner because it avoids mixed logic/power-level assumptions. For a rugged field build, an isolated transceiver is preferable when cable lengths, lightning exposure, or ground-potential differences justify the extra cost. [12]

The earlier "A and B tied together" loopback test should not be used as a pass/fail test for the UART-to-RS-485 path. RS-485 receivers detect the differential voltage between A and B; shorting the pair

removes the intended differential signal. Validate the interface using a known Modbus slave, a second transceiver, or an oscilloscope/logic analyzer at DI/RO and A/B.

8.4 GNSS: DFRobot TEL0157

CONFIRMED TEL0157 has communicated successfully with the controller in project diagnostics.

DFRobot TEL0157 uses a Quectel L76K GNSS module, supports GPS, GLONASS, and BeiDou, and exposes both I2C and UART. DFRobot lists 3.3-5.5 V operation, about 40 mA power consumption, a 32-channel receiver, and nominal 2.0 m CEP position accuracy. [6]

- Mount the GNSS antenna near the top of the enclosure with clear sky visibility.
- Keep the antenna and feed away from switching regulators, ESP32 antenna area, and noisy RS-485 cable bundles.
- Treat a zero position/time result during indoor bench testing as “no fix yet,” not automatically as communication failure.
- If power budget is tight, define a policy such as “acquire valid time/position at boot and periodically thereafter” rather than leaving GNSS continuously powered.

8.5 UV sensing

The original V4.0 reference used SI1145. Adafruit now marks that breakout as no longer stocked and notes that SI1145 does not contain a true UV sensing element; it estimates UV index from visible and infrared measurements. Adafruit recommends newer true-UV alternatives for new designs. [8]

The GUVA-S12SD option considered in this project uses a real UV-sensitive photodiode and produces an analog signal. DFRobot’s SEN0162 implementation specifies 200-370 nm detection, a 0-1 V output corresponding to a 0-10 UV-index mapping, very low operating current, and ±1 UV-index test accuracy. [9]

Option	Strengths	Trade-offs	Recommendation
SI1145	Simple I2C; direct UV-index estimate; established Arduino examples	Discontinued/no longer stocked by Adafruit; inferred UV rather than true UV element	Keep support only for legacy/reference builds.
GUVA-S12SD	True UV-sensitive photodiode; low current; simple analog interface	Requires ADC calibration and careful mechanical exposure; UV-index mapping depends on module/calibration	Good low-cost default if calibrated with the chosen ESP32-S3 ADC path.
Modern digital UV sensor	Digital interface, potentially easier calibration and reproducibility	Adds another device choice and driver	Offer as an alternative profile after core build is stable.

Mechanical warning: many clear plastics strongly attenuate UV. Do not place a UV sensor behind an arbitrary acrylic/polycarbonate “window” and assume the reading is valid. The optical path must be designed and validated.

8.6 Temperature, humidity, and pressure

TBD The exact environmental sensor for the present build is not fixed in the available project history.

The reference V4.0 uses BME280 for temperature/humidity/pressure and DS18B20 for an external temperature probe. These remain reasonable reference options, but the public repository should choose and test one exact baseline part before release. [1]

- Mount air temperature/humidity sensing in the ventilated metrology chamber, not beside the battery, DC/DC converter, or ESP32 regulator.
- Use a radiation shield / louver geometry to prevent direct solar heating while preserving airflow.
- If an external probe is required, document whether it measures air, surface, soil, or water temperature; these are not interchangeable measurements.

8.7 Optional sensors inherited from the reference design

BH1750 lux, PMS5003/7003 particulate matter, SHT10 soil sensing, and a tipping-bucket rain gauge can be added, but they should not be allowed to delay the first reproducible release. Add each as a self-contained hardware/driver profile with explicit power cost and enclosure implications. [1]

8.8 Power subsystem: Waveshare Solar Power Manager (D)

CONFIRMED Waveshare Solar Power Manager (D) is part of the current power-integration planning.

Waveshare describes Solar Power Manager (D) as compatible with 6-24 V solar panels, capable of charging a 3.7 V rechargeable lithium battery from solar or Type-C, and providing a regulated 5 V output up to 3 A. The Type-C interface is used for recharging/output on this product family, while dedicated 5 V output connections are available for powering the load. [7]

- For the station, prefer a documented dedicated 5 V output connection to the ESP32-S3 power input rather than relying on ambiguous cable behavior.
- Use a protected, reputable Li-ion/LiPo cell and mechanically restrain it inside the lower power enclosure.
- Route solar, battery, and load wiring separately enough that service work is obvious and polarity mistakes are difficult.
- Fuse/protect the battery path as appropriate; never treat a 3D-printed enclosure as the only lithium-battery safety control.
- Measure quiescent current of the complete power path, not only the ESP32-S3.

9. Mechanical and Enclosure Design

9.1 Existing design direction

The current enclosure concept uses a custom faceted/triangular geometry with three louvers on the long edges, a separate lower container for battery and power modules, and a removable upper enclosure. Hexagonal ventilation features, cable routing, and snap/click joining methods have been explored in the project conversations.

9.2 Functional zoning

Zone	Contents	Key rule

Metrology / ventilated zone	Air temperature, humidity, pressure; possibly light/UV depending geometry	Maximize representative airflow while excluding direct rain and solar radiation.
Control electronics zone	ESP32-S3, RS-485 interface, signal conditioning	Keep accessible and dry; avoid becoming a heat source for the metrology zone.
Power zone	Battery, solar manager, fuse/protection, main switch	Physically separate from metrology; provide strain relief and safe service access.
External instrument zone	Wind sensors, rain gauge, solar panel, GNSS antenna	Use weather-resistant mounting and cable entry; make replacement possible without opening every chamber.

9.3 Ventilation and rain control

- Louvers are preferable to simple vertical through-holes because they can block direct rain while allowing airflow.
- Hexagonal vents are printable, but they should be backed by a rain baffle or staggered path; do not place open vent holes directly above the battery/power electronics.
- Add insect mesh only where it does not substantially restrict airflow.
- Use white or light exterior surfaces around the air-sensor shield to reduce solar heating.

9.4 Removable upper enclosure joint

For a field instrument, a pure press-fit snap can become brittle after repeated UV/heat cycles. A better compromise is a locating tongue-and-groove perimeter plus two flexible side snap hooks for alignment, with one or two captive screws or quarter-turn fasteners as the positive retention method. This is still easy to remove but is less likely to open in wind or after material creep.

- Use draft/chamfer on mating rails to avoid scraping during assembly.
- Allow print tolerances; start around 0.3-0.5 mm clearance per side for FDM mating features and tune after test prints.
- Keep snap arms replaceable if possible; do not integrate a thin critical latch into the largest enclosure part if a small replaceable clip can do the job.
- Use a gasket only on electronics compartments that need sealing; the sensor chamber itself needs controlled ventilation.

9.5 Material recommendation

For long-term outdoor use, PETG or ASA is preferable to PLA. The reference V4.0 author also recommended ABS/PETG instead of PLA after using PLA for a quick prototype. [2]

9.6 Cable management

- Use keyed connectors or labeled terminal blocks rather than loose Dupont jumpers in the final outdoor build.
- Add printed cable combs, tie-down slots, or adhesive tie mounts on flat interior walls.
- Keep RS-485 twisted pair separate from high-current switching loops where practical.
- Provide drip loops outside the enclosure and strain relief at every cable entry.
- Photograph and diagram the final cable routing for the repository.

10. Firmware and Data Architecture

10.1 Recommended firmware structure

The firmware should be designed as independent drivers and services. Avoid a single `main.ino` that directly reads every register, formats every cloud message, and controls every power rail. The objective is to make hardware substitutions possible without rewriting the full station.

```
firmware/esp32s3/
  src/
    main.cpp
    board_config.h
    sensors/
      rs485_wind_speed.*
      rs485_wind_direction.*
      tel0157_gnss.*
      uv_sensor.*
      env_sensor.*
    services/
      modbus_bus.*
      power_manager.*
      data_logger.*
      network_client.*
      health_monitor.*
  test/
  platformio.ini
```

10.2 Common measurement record

```
{
  "timestamp_utc": "2026-08-14T06:00:00Z",
  "position": {"lat": null, "lon": null, "fix": false},
  "wind_speed_mps": 3.4,
  "wind_direction_deg": 135.0,
  "temperature_c": 29.6,
  "humidity_pct": 71.2,
  "pressure_hpa": 1007.8,
  "uv_index": 4.1,
  "battery_v": 4.02,
  "solar_v": 9.6,
  "status": {"rs485": "ok", "gnss": "no_fix"}
}
```

The repository should define this schema (or its final equivalent) before dashboard work. This prevents every UI from inventing its own field names and units.

10.3 Configuration policy

- Keep Wi-Fi credentials, API tokens, and private endpoints out of Git.
- Commit `config.example.h` or an equivalent template with safe placeholders.
- Make Modbus slave IDs, serial settings, register maps, sampling intervals, and sensor enable/disable flags configuration data rather than magic constants spread through the code.
- Record firmware version in every data packet/log file for traceability.

10.4 Low-power behavior

ESP32-S3 supports deep sleep, but deep sleep must be designed around measurement requirements. Wind speed and rain are event/time-dependent quantities; aggressively sleeping the MCU can cause loss of pulses or temporal resolution unless the sensor has its own register accumulation. By contrast, Modbus instruments that internally maintain the measurement can be polled periodically without continuously awake pulse counting. [10]

11. RS-485 / Modbus Engineering Lessons

11.1 Lesson: distinguish UART loopback from RS-485 bus validation

UART TX/RX loopback and RS-485 A/B are different layers. A UART loopback connects digital transmit data back to digital receive data. RS-485 A/B is a differential bus. Shorting A and B together does not create a valid differential loopback and can lead to a fail-safe/undefined receive state rather than echoed data. Use a second transceiver or a real Modbus slave for bus-level validation.

11.2 Lesson: address management is part of system architecture

The wind-direction work demonstrated that a sensor can be read correctly and that changing the Modbus ID becomes important once multiple instruments share the same cable. The repository should include an address-assignment procedure and a recovery procedure for devices accidentally configured to the same ID.

11.3 Recommended RS-485 bring-up checklist

10. Confirm sensor supply voltage and common reference requirements.
11. Confirm A/B polarity from the sensor manufacturer; naming conventions can differ.
12. Confirm baud rate, data bits, parity, and stop bits.
13. Poll a known register with one sensor only.
14. Record raw request and response frames in hex.
15. Validate scaling/units against the sensor display or controlled physical condition.
16. Assign a unique slave ID and power-cycle to verify persistence.
17. Add the second sensor and repeat before extending the cable.
18. Only after stable communication, add termination/biasing/protection based on the actual cable topology.

12. Power and Solar Autonomy Engineering

12.1 Power budget method

Do not size the battery from the ESP32 datasheet alone. Build an average-energy model that includes active MCU current, Wi-Fi transmit bursts, GNSS acquisition, RS-485 sensors, UV/environmental sensors, regulator losses, solar-manager quiescent current, and any always-on LEDs or USB bridges.

```
Daily energy (Wh/day) ≈ Σ [Voltage × Current × hours active per day]
Required battery Wh ≈ Daily energy × autonomy days / usable battery fraction
Required solar Wh/day ≈ Daily energy / (charging + conversion efficiency) × weather margin
```

12.2 Recommended first autonomy target

Design the first outdoor prototype for at least 3 days of operation without useful solar input. This is a planning target, not a guaranteed runtime claim. After measuring the real current profile, increase the battery/solar margin if the station will be deployed through long cloudy periods.

12.3 High-value power optimizations

- Disable or remove unnecessary power LEDs.
- Power-gate sensors that do not need to remain on continuously.
- Batch network transmissions rather than reconnecting for every single measurement if the use case allows.
- Use GNSS periodically rather than continuously if location is fixed and only time synchronization is needed.
- Measure complete-system sleep current with the final development board and power module.
- Store data locally before Wi-Fi attempts so repeated network retries do not waste energy and lose measurements.

12.4 Solar-panel placement

- Keep the solar panel thermally separated from the air-sensor chamber.
- Angle and orient the panel for the deployment location rather than mounting purely for appearance.
- Use a drip-resistant cable path and strain relief.
- Log battery and solar voltage/current if possible; power telemetry is essential for diagnosing field failures.

13. Budgeting and Procurement Strategy

The table below is a planning allowance for a one-off prototype, not a quotation. Prices vary strongly by sensor grade, shipping, supplier, and whether parts are already available. The purpose is to prevent under-budgeting the “small” items - connectors, cable glands, fasteners, spare modules, filament, and protection parts - that often dominate the final prototype cost.

Category	Typical planning allowance (USD)	Notes
ESP32-S3 dev board	8-20	Exact board choice affects sleep current, pin access, and USB convenience.
RS-485 wind speed sensor	25-60	Generic industrial Modbus class; exact SKU TBD.
RS-485 wind direction sensor	25-60	Generic industrial Modbus class; exact SKU TBD.
RS-485 interface + protection	5-30	Low end: basic transceiver module. High end: isolated/protected interface.
TEL0157 GNSS + antenna	20-35	Depends on supplier/bundle.
UV sensor	7-20	GUVA-S12SD class or modern digital alternative.

Temp/humidity/pressure sensor	5-25	BME280-class through higher-grade alternatives.
Rain gauge (optional first release)	15-40	Can be deferred if not yet integrated.
Waveshare Solar Power Manager (D)	20-35	Planning band; confirm current supplier price before purchase.
Solar panel	15-35	Size after measured energy budget.
Battery + protection/holder	15-35	Use reputable protected cells/battery pack.
Perfboard / prototype PCB / terminals	10-30	Prototype phase; custom PCB later.
Cable glands, connectors, wire, fasteners	20-45	Outdoor reliability budget.
3D-print material + failed iterations	10-30	PETG/ASA preferred for outdoor final parts.
Contingency / spare sensors	20-50	Highly recommended for a development project.

13.1 Phase-level budget

Phase	Indicative total	What it includes
P0 bench prototype	$100-180	Controller, core sensors, RS-485, GNSS, basic power path, perfboard/wiring; may omit outdoor mechanical parts.
P1 outdoor prototype	$180-300	Adds solar/battery sizing, printed enclosure, weatherproof connectors, full sensor set, spares.
P2 ruggedized engineering build	$300-450+	Isolation/surge protection, higher-grade connectors/sensors, custom PCB, calibration/validation accessories.

Tools such as a 3D printer, soldering station, multimeter, bench supply, oscilloscope/logic analyzer, crimp tools, and computer are excluded because they are reusable capital tools rather than per-unit BOM items.

13.2 Procurement policy for the repository

- Every BOM item should have manufacturer, manufacturer part number, quantity, electrical function, acceptable substitute, and supplier example.
- Avoid vendor-only links as the sole identification method; links can disappear.
- For generic marketplace sensors, archive the protocol/register manual in `docs/datasheets/` only if redistribution is legally permitted; otherwise store a source link and a local notes file.
- Buy at least one spare of low-cost critical modules during development.

14. Build, Integration, and Validation Procedure

14.1 Stage A - bench electronics

19. Power ESP32-S3 from a known bench/USB supply.
20. Bring up one sensor at a time with a minimal diagnostic firmware.
21. Create a standalone RS-485 scan/read diagnostic and confirm raw frames.
22. Confirm TEL0157 communication first, then obtain an outdoor fix.
23. Characterize UV ADC response under repeatable light conditions.
24. Integrate sensor modules only after each passes its individual test.

14.2 Stage B - power path

25. Connect the protected battery to the solar manager with solar disconnected.
26. Verify regulated load output and ESP32 boot.
27. Verify Type-C charging behavior separately.
28. Connect solar panel and verify charge indicators/telemetry.
29. Measure current in active, network transmit, sensor-poll, GNSS-acquisition, and sleep states.
30. Run a battery-only endurance test and compare measured runtime with the energy model.

14.3 Stage C - enclosure integration

31. Install cable glands and strain relief before final sensor wiring.
32. Keep power module and battery in their dedicated lower compartment.
33. Mount metrology sensors in the ventilated region away from heat sources.
34. Mount GNSS antenna for sky view.
35. Perform repeated assembly/removal cycles on the upper enclosure joint.
36. Perform a controlled spray/rain test without powered lithium electronics first; inspect for ingress paths and trapped water.

14.4 Stage D - field validation

37. Run at least 72 h outdoors with logging enabled.
38. Compare temperature/humidity/pressure and UV against a trusted nearby reference, while recognizing that microclimate differences are real.
39. Rotate wind direction sensor to known compass headings and verify output.
40. Check wind-speed plausibility and zero behavior.
41. Review battery trend over day/night cycles.
42. Force Wi-Fi outage and verify that data collection continues/recovery is clean.
43. Power-cycle during operation and verify automatic recovery.

15. Risks and Failure Modes

Risk	Effect	Mitigation / test
Solar heat biases air sensor	Temperature/humidity readings become systematically wrong	Physical thermal separation, white shield, airflow, comparison testing.

Water enters through vents/cable entries	Corrosion, shorts, battery hazard	Louver/baffle geometry, glands, drip loops, spray test, conformal coating where appropriate.
MAX485 mixed-voltage integration	Unreliable or electrically unsafe logic assumptions	Use native 3.3 V transceiver or explicit level handling; document supply/logic levels.
RS-485 A/B naming mismatch	No sensor response	Document actual polarity from working build; provide swap diagnostic.
Duplicate Modbus IDs	Bus collisions / wrong responses	Repository address-assignment tool and label each sensor with configured ID.
GNSS antenna obstructed	Long or failed fix acquisition	Top placement, outdoor test, diagnostic LEDs/status.
UV window attenuates UV	False low UV readings	Validate optical material or expose sensor through a protected UV-transparent path.
Battery undersized	Night/cloud shutdowns	Measured energy budget, 3-day initial autonomy target, brownout/recovery test.
Dev-board sleep current too high	Solar budget fails despite low MCU datasheet current	Measure entire board/system; remove LEDs/USB bridge or migrate to custom PCB if necessary.
Snap latch fatigue	Enclosure opens or becomes hard to service	Use locating geometry plus captive positive fastener; make clip replaceable.
Repository cannot be reproduced	Open-source value is lost	Versioned BOM/CAD/firmware, release artifacts, test logs, build-from-clean checkout verification.

16. Improvement Roadmap

Development Roadmap

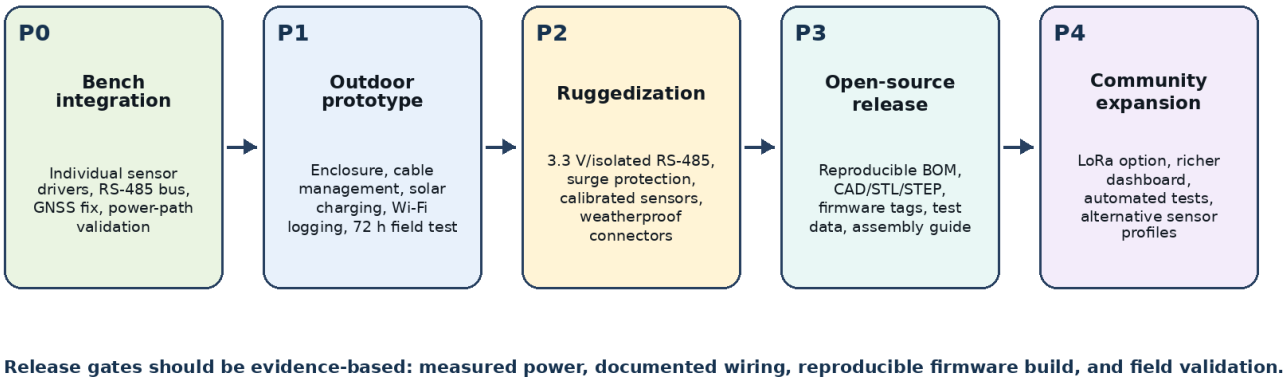


Figure 2. Recommended staged roadmap from bench integration to community-ready release.

16.1 Highest-priority improvements

- Finalize the exact baseline sensor list and part numbers.
- Replace development-only Dupont wiring with keyed/locking connectors and proper terminals.
- Move from MAX485 module to a 3.3 V or isolated/protected RS-485 interface.
- Measure the complete energy budget and size solar/battery from data.
- Finalize the upper/lower enclosure joint and weatherproof cable entry.
- Create a single firmware branch with modular drivers and a defined data schema.
- Perform and publish a 72 h outdoor test before calling the build “v1.0.”

16.2 Medium-term improvements

- Custom carrier PCB for ESP32-S3, RS-485 protection, switched sensor rails, battery/solar telemetry, and keyed connectors.
- Optional LoRa sender/receiver profile inspired by the V4.0 reference, kept separate from the core build.
- On-device storage (microSD or flash ring buffer) for network outages.
- Automated dashboard provisioning and MQTT discovery.
- Calibration profiles stored per sensor serial number or Modbus ID.
- CI checks that compile firmware and validate documentation links/JSON schema.

16.3 Longer-term research directions

- Solar irradiance/PAR sensing for renewable-energy or agriculture studies.
- Air-quality integration with inlet design that does not compromise weatherproofing.
- Fleet management for multiple stations using unique station IDs and remote configuration.
- OTA firmware updates with rollback and versioned configuration migrations.
- Comparative validation against a reference weather station across seasons.

17. GitHub Repository Blueprint

17.1 Proposed repository structure

```
solar-weather-station/
├── README.md
├── LICENSES/
│   ├── SOFTWARE-LICENSE.txt
│   └── HARDWARE-LICENSE.txt
├── CONTRIBUTING.md
├── CHANGELOG.md
├── CITATION.cff
├── SECURITY.md
├── docs/
│   ├── 01-problem-and-objectives.md
│   ├── 02-system-architecture.md
│   ├── 03-assembly-guide.md
│   ├── 04-wiring-and-pinout.md
│   ├── 05-modbus-registers.md
│   ├── 06-power-budget.md
│   ├── 07-calibration.md
```

```
│   ├── 08-test-results.md
│   └── decisions/
├── firmware/
│   └── esp32s3/
├── hardware/
│   ├── wiring/
│   ├── pcb/
│   ├── rs485/
│   └── power/
├── mechanical/
│   ├── source-cad/
│   ├── step/
│   ├── stl/
│   └── print-settings.md
├── bom/
│   ├── bom.csv
│   └── substitutions.md
├── data/
│   ├── schema/
│   └── sample/
├── tools/
│   ├── modbus_id_tool/
│   └── diagnostics/
├── tests/
│   ├── bench/
│   └── field/
└── .github/
    ├── ISSUE_TEMPLATE/
    └── workflows/
```

17.2 README should answer these questions immediately

- What problem does the station solve?
- What exactly does the current release measure?
- What is required to build it?
- What is the expected cost band?
- How is it powered?
- How do the RS-485 sensors connect and what are their slave IDs?
- How do I flash firmware from a clean computer?
- How do I validate that each sensor works?
- What is not yet production-ready?
- Which files are original work and which are adapted/derived from external projects?

17.3 Engineering decision records

Create short Architecture Decision Records (ADRs) under `docs/decisions/`. Examples: “ADR-001 Use ESP32-S3,” “ADR-002 Use RS-485 for wind instruments,” “ADR-003 Select baseline UV sensor,” “ADR-004 Power architecture,” and “ADR-005 Enclosure fastening strategy.” This preserves design intent when community contributors propose alternatives.

17.4 Release assets

- Tagged firmware source and compiled binary.
- BOM CSV with MPNs and substitutions.
- Wiring diagram PDF/PNG/SVG.

- STEP and STL exports plus source CAD where possible.
- Print orientation/settings and material recommendation.
- Modbus register map and configured IDs.
- Power-current measurement table.
- At least one field-test data sample.
- Known issues and photos of the actual released hardware revision.

18. Open-Source Licensing and Attribution

This section is engineering guidance, not legal advice. Licensing must be resolved before the first public release because the project draws inspiration from external open designs and may reuse code.

18.1 Firmware

If firmware is copied or adapted from the GPL-3.0 `jhughes1010/weather_v4_lora` or receiver repositories, the distributed derivative code must comply with GPL-3.0 obligations. The safest route is to keep any directly adapted firmware under GPL-3.0 and preserve copyright/license notices. [3][4] [13]

If the project firmware is independently rewritten without incorporating GPL code, a permissive license such as Apache-2.0 can be considered. Apache-2.0 is a widely used open-source software license and includes explicit patent terms. [15]

18.2 Hardware and CAD

For original PCB, wiring, and mechanical design files, CERN Open Hardware Licence v2 is a strong fit for an open hardware project. CERN-OHL-P-2.0 is the permissive variant and is intended to support use, study, modification, sharing, and distribution of hardware designs/products. [14]

18.3 Documentation

Use a documentation/content license separately (for example CC BY 4.0) if desired. Do not assume a software license automatically communicates how photographs, CAD renders, or written instructions may be reused.

18.4 Attribution file

Add `NOTICE.md` or `THIRD_PARTY.md` containing the Open Green Energy V4.0 reference, the James Hughes transmitter/receiver repositories, sensor/vendor documentation, and any libraries reused by the firmware. This makes provenance visible and helps future contributors avoid accidental license mixing.

19. Release Definition of Done

- The exact ESP32-S3 board/module is identified and photographed.
- Every connected sensor has a manufacturer/model/part number and interface specification.
- Wind-speed and wind-direction Modbus IDs and register maps are documented.

- GNSS obtains a valid outdoor fix and the firmware clearly distinguishes no-fix from communication failure.
- UV sensing method is finalized and its calibration/limitations are documented.
- The complete station runs from solar-manager + battery power and recovers from power interruption.
- Active/sleep current measurements are published.
- Enclosure CAD, STEP/STL, material, print orientation, and fasteners are published.
- Cable routing, strain relief, and connector labels match the wiring diagram.
- A 72 h outdoor validation log is published with no unexplained resets/data gaps.
- Firmware builds from a clean checkout using documented commands.
- No credentials, tokens, Wi-Fi passwords, or private URLs are committed.
- LICENSE/NOTICE/third-party attribution is present and internally consistent.
- Known limitations are explicit; the README does not claim certified meteorological or IP performance.

Appendix A. Proposed Baseline BOM

Group	Item	Status	Qty	Notes
Controller	ESP32-S3 development board	Confirmed direction	1	Exact SKU to lock before v1.0.
Communications	3.3 V RS-485 transceiver module / isolated interface	Recommended	1	Replace MAX485 for final integration.
Wind	RS-485 Modbus wind speed sensor	Confirmed category	1	Record exact model, registers, scale.
Wind	RS-485 Modbus wind direction sensor	Confirmed category	1	Direction decoding already achieved.
Position/time	DFRobot TEL0157 GNSS + antenna	Confirmed	1	I2C/UART; GPS/GLONASS/BeiDou.
UV	GUVA-S12SD module or selected modern digital UV sensor	Decision pending	1	SI1145 legacy-only recommendation.
Environment	Temp/humidity/ pressure sensor	TBD	1	BME280 is the reference baseline option.
Rain	Tipping-bucket rain gauge	Optional	1	Can be deferred to v1.1.
Power	Waveshare Solar Power Manager (D)	Confirmed direction	1	6-24 V solar input, 3.7 V battery, regulated 5 V load.
Power	Protected 3.7 V Li-ion/LiPo battery	Required	1	Capacity after measured power budget.
Power	Solar panel	Required	1	Voltage/power sized after measurement.
Mechanical	Custom printed enclosure set	In development	1	PETG/ASA preferred.
Mechanical	Cable glands / strain relief / fasteners	Required	set	Weather-resistant field integration.
Prototype	Perfboard / prototype PCB / terminals	Current prototyping	set	Transition to carrier PCB later.

Appendix B. Interface Plan

Interface	Device(s)	Electrical notes	Firmware owner
UART + DE/RE	RS-485 transceiver → wind sensors	One differential bus, unique Modbus IDs; use 3.3 V logic transceiver	`modbus_bus` + wind drivers
I2C or UART	TEL0157 GNSS	Choose one interface and document switch/pin state;	`tel0157_gnss`

		I2C can share bus if addresses are compatible	
ADC	GUVA-S12SD	Analog output; characterize ESP32-S3 ADC scaling and supply/reference	`uv_sensor`
I2C	Temp/humidity/pressure	Exact sensor TBD; keep bus pullups and address map documented	`env_sensor`
GPIO interrupt / counter	Rain gauge if added	Debounce and persistence strategy required	`rain_sensor`
ADC / monitor	Battery and solar telemetry if exposed	Voltage-divider calibration and protection required	`power_manager`
Wi-Fi	Network/cloud	No secrets in source; retry/backoff and local queue	`network_client`

Exact ESP32-S3 GPIO numbers should be assigned only after the final development-board SKU and all reserved pins are known. The public repository should include a pin-map table generated from the firmware configuration.

Appendix C. Test Matrix

Test ID	Test	Pass condition	Evidence to commit
T-001	ESP32-S3 clean build/flash	Fresh checkout compiles and flashes without manual source edits	CI log + build instructions
T-010	RS-485 wind speed	100 consecutive valid polls; no CRC/timeout errors under bench wiring	Serial log + raw frames
T-011	RS-485 wind direction	Known headings map to correct degrees/directions	Heading test table
T-012	Modbus ID change	New ID persists after power cycle; no collision with second sensor	Tool output + config file
T-020	GNSS communication	Module detected and data fields readable	Diagnostic log
T-021	GNSS outdoor fix	Valid UTC date/time and non-zero position outdoors	Diagnostic log + location redaction guidance
T-030	UV response	Output changes monotonically with controlled UV exposure; calibration stored	Calibration CSV/plot
T-040	Power output	ESP32 runs from solar manager/battery path	Voltage/current table
T-041	Sleep profile	Measured sleep/active current documented	Current measurement table
T-042	Battery-only endurance	Runtime meets or exceeds engineering target	Endurance log

T-050	Enclosure assembly cycles	20 open/close cycles without latch damage or loss of fit	Photos + notes
T-051	Water ingress	No water reaches protected electronics after defined spray test	Test procedure + photos
T-060	Network outage	Data collection continues; reconnect succeeds without reset loop	Event log
T-070	72 h outdoor test	No unexplained resets, bus lockups, or severe sensor anomalies	Field CSV + test report

Appendix D. README Starter Outline

- Project one-line description and current hardware photo.
- Why this project exists / problem statement.
- Current release status and explicit “experimental / not certified” note.
- Features and measured variables.
- Architecture diagram.
- Hardware BOM and cost band.
- Wiring and Modbus IDs.
- Mechanical assembly / 3D printing.
- Firmware prerequisites and clean build steps.
- Configuration and secrets handling.
- Sensor validation and calibration.
- Power budget and measured runtime.
- Field test results.
- Known issues / roadmap.
- Reference projects and attribution.
- Licenses and contribution guide.

References

[1] Open Green Energy, “Solar Powered WiFi Weather Station V4.0,” Hackaday.io. https://hackaday.io/project/187061-solar-powered-wifi-weather-station-v40

[2] Open Green Energy, V4.0 project logs/components/files, Hackaday.io. https://hackaday.io/project/187061/logs

[3] James Hughes, `jhughes1010/weather_v4_lora`, GitHub, GPL-3.0. https://github.com/jhughes1010/weather_v4_lora

[4] James Hughes, `jhughes1010/weather_v4_lora_receiver`, GitHub, GPL-3.0. https://github.com/jhughes1010/weather_v4_lora_receiver

[5] teamsuperpanda, `weather-station`, GitHub archive; ESPHome adaptation of Open Green Energy V4.0. https://github.com/teamsuperpanda/Weather-Station

[6] DFRobot, “Gravity: GNSS GPS BeiDou Receiver Module (TEL0157),” official wiki. https://wiki.dfrobot.com/tel0157/

[7] Waveshare, “Solar Power Manager (D),” official wiki/product documentation. https://www.waveshare.com/wiki/Solar_Power_Manager_%28D%29

[8] Adafruit, “SI1145 Digital UV Index / IR / Visible Light Sensor,” product documentation. https://www.adafruit.com/product/1777

[9] DFRobot, “Gravity: GUVA-S12SD Analog UV Light Sensor (SEN0162),” official wiki. https://wiki.dfrobot.com/sen0162/

[10] Espressif, “Sleep Modes - ESP32-S3,” ESP-IDF Programming Guide. https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/sleep_modes.html

[11] Espressif, “ESP32-S3 Wi-Fi & BLE 5 SoC,” product overview. https://www.espressif.com/en/products/socs/esp32-s3

[12] Analog Devices, MAX485 product information and RS-485 implementation resources. https://www.analog.com/en/products/max485.html

[13] GNU Project, GNU General Public License v3 information. https://www.gnu.org/licenses/gpl-3.0.html

[14] SPDX, CERN Open Hardware Licence Version 2 - Permissive (CERN-OHL-P-2.0). https://spdx.org/licenses/CERN-OHL-P-2.0.html

[15] Apache Software Foundation, Apache License 2.0. https://www.apache.org/licenses/LICENSE-2.0

[16] Elecrow, “DIY Solar Powered WiFi Weather Station V4.0,” reference page used in the initial project discussion. https://www.elecrow.com/sharepj/diy-solar-powered-wifi-weather-station-v40-802.html

Project-internal source trail

This foundation also incorporates the available Solar Weather Station design conversations in the Green Energy Projects workspace dated 26 May 2026, 15 June 2026, 17 June 2026, and 2 July 2026. Those sessions contain the project-specific decisions on ESP32-S3 layout, MAX485/RS-485 diagnostics, TEL0157 bring-up, wind-direction decoding and Modbus addressing, and the custom 3D-printed enclosure/power-module integration.

Version note

Version 0.1 is a consolidation baseline. Before publishing it as the definitive v1.0 project document, replace all “TBD” items with exact part numbers, insert actual wiring/pin assignments, attach measured power data, and update the BOM from the hardware that is physically built.